

BY Rewards Membership Program

Architecture & Design Patterns Guide

Java 21 | Spring Boot 4 | Gradle
Self-contained demo with swappable infrastructure

Generated from by-reward-core codebase

1. High-Level Architecture

This backend is a modular monolith built with Spring Boot 4 and Java 21. It is organized by feature (package-by-feature) with a hexagonal-lite style: domain logic lives in services, infrastructure sits behind swappable adapters, and REST controllers remain thin.

Core architectural idea

One deployable application that behaves like a distributed system: events, outbox, caching, and resilience patterns are all present. It runs locally with H2 file DB, Caffeine cache, and in-process Spring events. Postgres, Redis, and Kafka can replace these without changing domain code.

Request flow (simplified)

```
REST Client / Swagger
-> Controllers (/api/v1/**)
-> Services (business logic)
-> Repositories (JPA) + Cache (Caffeine)
-> H2 file database (./data/membership.mv.db)
```

2. Package Structure (Package-by-Feature)

- plan/ - Membership plans (Monthly / Quarterly / Yearly)
- tier/ - Tiers and configurable progression rules
- benefit/ - Benefit catalog and per-tier perks
- discount/ - Plan-scoped discount eligibilities (product/category)
- deal/ - Deals and plan-based early access
- subscription/ - Subscribe, upgrade, downgrade, cancel, history
- member/ - Member profile and composite membership view
- event/ - Domain events, outbox, order ingestion
- cache/ - Cache reload APIs
- common/ - Shared exceptions, web, concurrency helpers
- config/ - Spring configuration, seeding, OpenAPI
- observability/ - Metrics listeners

Each feature package typically contains:

domain/	JPA entities, enums, domain rules
repository/	Spring Data JPA
service/	Business logic
dto/	Request/response records
web/	REST controllers

3. Domain Model

Key concepts

- MembershipPlan - what the user pays for (Monthly / Quarterly / Yearly)
- Tier - loyalty level (Silver / Gold / Platinum)
- Subscription - links user to plan + tier for a time period
- BenefitDefinition + TierBenefit - configurable perks per tier
- BenefitEligibility - plan-scoped discount rules (ALL / PRODUCT_CATEGORY / PRODUCT_ITEM)
- Deal + DealPlanAccess - exclusive deals and early access per plan
- Member + UserOrderStats - profile and rolling order statistics
- SubscriptionHistory - audit trail of tier and status changes
- OutboxEvent - transactional outbox for reliable event delivery

Important separation

- Plan drives discounts and deals (commercial subscription benefits)
- Tier drives loyalty benefits and auto-progression (Silver / Gold / Platinum)
- Subscription connects user + plan + tier + expiry

4. Design Patterns

Facade - MembershipFacade

Composes member profile, active subscription, plan, tier, and resolved benefits into a single MembershipView. Keeps controllers thin.

State - SubscriptionStateMachine

Guards subscription lifecycle transitions (SUBSCRIBE, UPGRADE, DOWNGRADE, CANCEL, EXPIRE). Illegal transitions are rejected consistently.

Strategy + Specification - TierEvaluationService

RuleCriterion beans (MinOrders, MonthlyOrderValue, Cohort) evaluate TierRule rows. Adding a criterion = adding a Spring bean.

Chain of Responsibility - BenefitResolverChain

BenefitResolver beans transform TierBenefit rows into EffectiveBenefit DTOs. Add a benefit type = add a resolver bean.

Strategy - EligibilityMatcher

Matches discount scopes (ALL, PRODUCT_CATEGORY, PRODUCT_ITEM). Most-specific-wins: item > category > all.

Ports & Adapters - EventPublisher

Port interface with OutboxEventPublisher adapter. Swap to Kafka without changing producers.

Repository - Spring Data JPA

All persistence through repositories with optimistic locking and unique constraints.

Builder / Factory

Entity builders, SubscriptionHistory builder, domain event records.

Transactional Outbox

Events persisted in OutboxEvent table in same transaction, then published after commit.

5. Event-Driven Tier Progression

Order events drive automatic tier upgrades/downgrades:

```
POST /api/v1/events/orders
-> OrderIngestionService
-> EventPublisher.publish(OrderPlacedEvent)
-> OutboxEvent saved (same DB transaction)
-> Spring ApplicationEventPublisher
-> OrderPlacedListener (async, after-commit, virtual threads)
-> OrderStatsService.applyOrder (rolling stats, @Version)
-> TierEvaluationService.evaluateBestTier
-> SubscriptionService.applyAutoTierChange (if tier changed)
-> MemberTierChangedEvent published
```

Why this design

- Order ingestion stays fast (202 Accepted)
- Stats and tier evaluation run asynchronously
- Outbox guarantees events are not lost on crash
- Virtual threads keep async concurrency cheap

6. Caching Architecture

Read-heavy configuration is cached with Caffeine:

- plans - Active membership plans
- tiers - Active tiers with benefits
- tierRules - Progression rules
- benefitEligibilities - Plan discount scopes
- deals - Active deal catalog + plan access

Writes evict caches (@CacheEvict). Cache reload APIs at /api/v1/cache/* allow refresh without restart.

7. Concurrency & Resilience

Concurrency mechanisms

- Optimistic locking (@Version) on Subscription and UserOrderStats
- OptimisticRetry with REQUIRES_NEW TransactionTemplate per retry attempt
- IdempotencyRecord for safe subscribe retries (Idempotency-Key header)
- DB unique constraint: one active subscription per user
- Virtual threads for async event listeners

Resilience4j

- Rate limiter catalogRead - plan/tier/deal/discount GET APIs
- Rate limiter + bulkhead subscriptionWrite - subscribe, upgrade, downgrade
- Circuit breaker tierEvaluation - protects order event pipeline

8. API Layer & Cross-Cutting Concerns

- DTOs as Java records - immutable, lean JSON
- Bean Validation with meaningful field-level error messages
- GlobalExceptionHandler returns uniform ApiError (timestamp, status, message, path, traceId, details)
- CorrelationIdFilter injects X-Correlation-Id into MDC for tracing
- Micrometer + Prometheus metrics; EventMetricsListener counts domain events
- ECS structured JSON logs to logs/membership.json

9. Key Business Flows

Discounts (product listing page)

GET /api/v1/members/{userId}/discounts returns all discount badges for the user's plan, grouped by scope (allDiscounts, categoryDiscounts, itemDiscounts). Frontend indexes once and annotates every product without per-item API calls.

YEARLY plan example:

ALL: 5% off all items

PRODUCT_CATEGORY ELECTRONICS: 15%

PRODUCT_ITEM SKU-IPHONE: 25%

Cart validation: GET /members/{userId}/discount?productId=&categoryId=

Deals (plan-based early access)

Deals are tied to membership plans via DealPlanAccess. GET /api/v1/deals lists full catalog with per-plan access schedule. GET /api/v1/members/{userId}/deals returns deals accessible to the member now.

Tier history

GET /api/v1/members/{userId}/tier-history returns all tier-change events (SUBSCRIBE, UPGRADE, DOWNGRADE, TIER_AUTO_CHANGE) with resolved tier codes.

10. Extensibility & Infrastructure Swapping

- New billing plan: Insert MembershipPlan or POST /api/v1/plans
- New tier: Insert Tier + TierBenefit rows
- New benefit type: Add BenefitType enum + BenefitResolver bean
- New progression rule: Add TierRule row + optional RuleCriterion bean
- New discount scope: Add EligibilityScopeType + EligibilityMatcher bean
- Kafka events: Implement EventPublisher adapter or relay OutboxRelay
- Redis cache: Replace CacheManager bean in CacheConfig
- Postgres DB: Change spring.datasource.url

Architectural principles

- Separation of concerns - package-by-feature, thin controllers
- Open/Closed - strategy beans for rules, benefits, discounts
- Dependency inversion - EventPublisher port, cache abstraction
- Config over code - plans, tiers, rules, eligibilities, deals are data-driven
- Eventual consistency - async tier evaluation after order events
- Fail safe - state machine, circuit breaker, optimistic retry

Appendix: API Summary

- GET /api/v1/plans
- POST /api/v1/plans
- GET /api/v1/tiers
- POST /api/v1/subscriptions
- GET /api/v1/subscriptions?userId=
- GET /api/v1/members/{userId}/membership
- GET /api/v1/members/{userId}/discounts (product listing)
- GET /api/v1/members/{userId}/discount (cart validation)
- GET /api/v1/members/{userId}/deals
- GET /api/v1/members/{userId}/tier-history
- GET /api/v1/benefit-eligibilities
- POST /api/v1/benefit-eligibilities
- GET /api/v1/deals
- POST /api/v1/deals
- POST /api/v1/events/orders
- GET /api/v1/cache/names
- POST /api/v1/cache/reload-all
- Swagger UI: /swagger-ui